第三章 SQL 注入漏洞与防护——实验分析报告

提交前自查声明（防红线）

- [x]我只在授权环境（本地 DVWA127.0.0.1）中完成实验。
- [x]我的所有截图均已完成关键隐私信息脱敏。
- [x]我已明确区分“正常查询变化”和“查询逻辑被改变”。
- [x]我已明确将根因归结为“结构被污染”。

任务 1：数据库与查询条件基础说明（3 分）

1.最小业务链路演示
以“电商网站搜索商品”为一个具体业务场景：

用户在前端输入框输入“手机”⟶后端接收输入并构造查询 SELECT * FROM products WHERE name = '手机'; ⟶数据库执行并返回对应结果⟶前端渲染商品列表。

2.页面输入为何与数据库有关？如何进入查询条件？
页面搜索功能的本质是动态数据检索。前端接收的用户输入（如“手机”），会被作为“变量”传递给后端代码（如 PHP、Java）。后端代码通常通过“字符串拼接”的方式，将这个变量直接拼接到预先写好的 SQL 语句的条件字句（WHERE）中，从而完成查询条件的构造。

3.为什么这为 SQL 注入创造了可能性？
因为系统采用了“字符串拼接”的机制，**只验证了数据能否传过来，却没有验证传过来的是“纯数据”还是“可执行的指令”**。当用户输入特定的符号（如单引号）时，会提前闭合原有的查询逻辑，导致数据库解析器将用户输入的一部分当作了 SQL 结构代码来执行，这就创造了注入的可能。

任务 2：DVWA SQL Injection Low 现象记录（4 分）

1.实验目标与页面

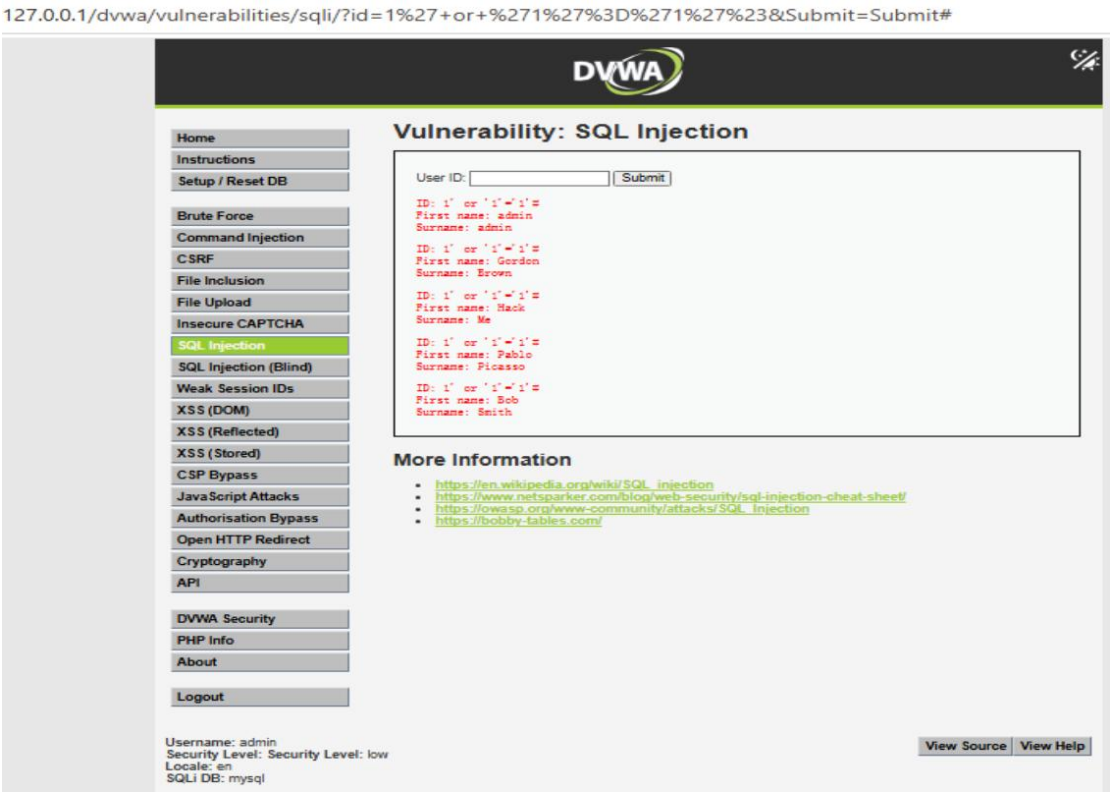
- 目标：本地 DVWA 环境下的 Vulnerability:SQLInjection 模块。
- 级别：Low。

2.普通输入与异常输入对比及结果差异

- **普通输入现象：**在 UserID 框输入 1。

- 分析：只返回了 ID 为 1 的单个用户（admin）的信息。符合正常业务查询逻辑。

- **异常输入现象**：在 UserID 框输入 1'or'1'='1'#。



- 分析：返回了数据库中所有用户的信息。

3.为什么这种差异不像“正常查询变化”？

- **这个差异支撑了我的判断：**正常的查询变化应该是“我搜 ID=2 就返回 ID=2 的数据”，是数据集合的切换；而此处的差异是“无视了业务限定条件，强制输出了本不该输出的全体数据”。这证明后台的 WHERE 限制条件被绕过，查询逻辑发生了本质改变。

任务 3：机制分析与差异比较（4.5 分）

1.机制因果链条分析（含 SQL 拼接前后对照）
用户输入恶意 payload →后端通过字符串拼接将其接入预设 SQL 语句：

- 拼接前的预期 SQL：SELECT first_name, last_name FROM users WHERE user_id = '1';
- 拼接后的污染 SQL：SELECT first_name, last_name FROM users WHERE user_id = '1' or '1'='1' #';

→数据库解析时，单引号闭合了原有的字符串边界→数据库将后续的 or '1'='1' 执行为恒真逻辑→导致回显或行为差异（返回所有数据）。

2.强制判断题表态

- **关于 SQL 注入的根因更接近：**
  - **我的选择：B.输入进入了不该进入的查询结构。**
  - **理由：**单引号或#等字符本身没有任何破坏力，只有当它们被以不安全的方式（拼接）送入解释器，改变了原有的抽象语法树（AST）结构时，才引发了漏洞。根因在于“结构被污染”。
- **关于盲注和普通注入的最大差异更接近：**
  - **我的选择：B.证据形式不同。**
  - **理由：**两者的根因完全相同（都是输入污染了查询结构）。差异仅在于，普通注入能直接看到数据库的脏数据（回显），而盲注由于业务屏蔽了回显，只能通过页面返回的 True/False 或响应时间的延迟（行为差异）来作为推断证据。

3.Low 与 Medium 的差异

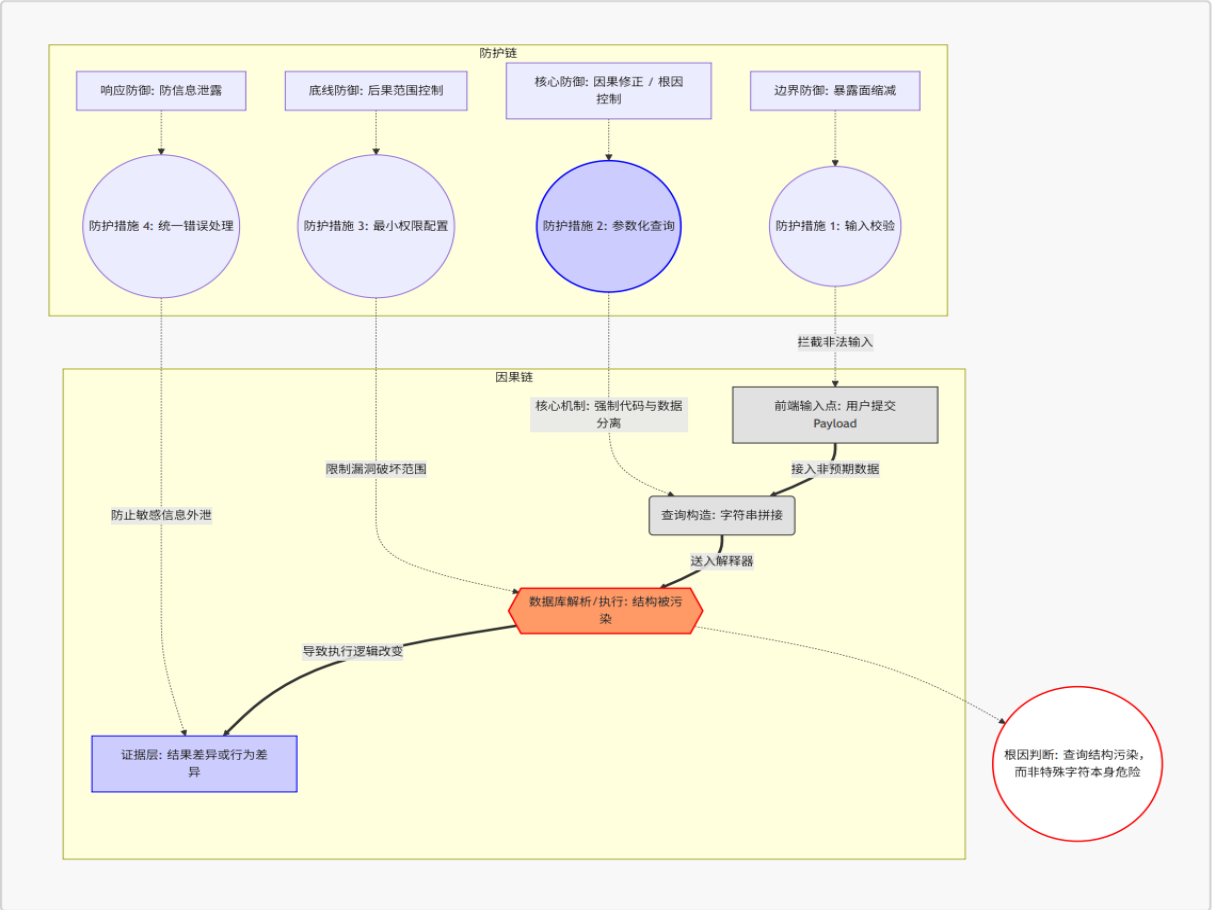
- **Low 级别：**毫无防备，直接接收前端参数并进行字符串拼接，导致系统的根因与暴露面完全处于敞开状态。
- **Medium 级别：**在控制思路上尝试了“暴露面缩减”，包括前端下拉菜单限制与后端转义（如 mysql_real_escape_string）。**但在真实的安全评估视角中，前端的下拉菜单限制形同虚设。攻击者只需使用 BurpSuite 等抓包代理工具拦截 HTTP 请求，即可轻易篡改参数，直接击穿前端防线** 此时防御压力全落在后端的转义函数上。由于底层的核心机制依然是危险的“字符串拼接”而未实现结构分离，一旦遇到无需单引号闭合的注入（如数字型），Medium 级别的防护体系即刻崩溃。

任务 4：SQL 注入防护清单与解释（3.5 分）

防护措施	解决什么问题	控制的是哪一层？	它不能替代什么？
参数化查询	解决数据被误解析为代码的问题，强制分离代码结构与数据参数。	**根因控制**	这是核心机制，但不能替代最小权限（万一业务逻辑有洞，仍需权限兜底）。
输入校验(白名单)	拦截非法格式的输入，确保输入符合业务预期（如 ID 只能是数字）。	**暴露面缩减**	不能替代参数化查询，因为合法的字符组合有时也可能被利用。
错误处理	屏蔽数据库详细报错信息（如表名、绝对路径），返回通用提示。	**暴露面缩减**(防信息泄露)	减少错误信息泄露，降低错误型注入的信息收集价值，但不能真正阻止布尔盲注或时间盲注。
最小权限	限制 Web 应用连接数据库的用户权限（如禁止 DROP、限制访问特定表）。	**后果范围控制**	不能阻止注入漏洞的产生，只是将系统被攻破后的损失降到最低。
***(补充)*WAF 防火墙**	通过流量规则匹配拦截常见的 SQL 注入特征。	**暴露面缩减**	不能替代安全编码，因为特征库总有被绕过（Bypass）的可能。

七、收口项

收口项 1：SQL 注入因果链与防护关系图（3 分）



上图清晰地映射了 SQL 注入的攻击发生链路与多维防护体系的对应关系：

1. **漏洞因果链与核心判定**图中的下半部分展示了注入发生的完整因果链：前端输入的 Payload 通过不安全的“字符串拼接”进入系统。当其被送入数据库解释器时，原有的抽象语法树被破坏。

- **根因判定（红色预警节点）：**SQL 注入的根本原因在于**查询结构被污染**，而非用户输入的某些特殊字符本身带有危险。如果没有拼接漏洞，字符永远只是纯数据。

2. **四维防护体系的精准拦截**针对上述因果链，图中的上半部分构建了四道防线，并精准卡位在漏洞流转的不同节点：

- **【核心防御-根因控制】参数化查询：**作用于查询构造阶段。这是最根本的防御手段，它通过强制机制实现了**代码与数据的严格分离**，直接掐断了“结构被污染”的可能性。
- **【边界防御-暴露面缩减】输入校验：**作用于最前端。作为第一道关卡拦截非法输入格式，减少系统面对攻击的暴露面，但无法替代核心防御。

- **【底线防御-后果控制】最小权限配置：**作用于数据库执行阶段。作为“托底”手段，假设漏洞已被触发，它能将攻击者的破坏范围死死限制在最低限度（例如无法删库、无法跨表）。
- **【响应防御-防信息泄露】统一错误处理：**作用于最终的证据结果层。屏蔽数据库底层报错，减少错误信息泄露，但需注意它无法真正阻止盲注。

总结：防护措施绝不是平级的，只有**参数化查询**对症下药解决了根因，其余三项均属于“纵深防御”体系中的暴露面缩减与后果控制手段。

收口项 2：学生自评表（2 分）

1. **我本章最清楚的一点是什么？**
   最清楚的是 SQL 注入的本质不是“输入了特殊字符”，而是“结构被污染”。如果使用参数化查询，再特殊的字符也只能当做死数据处理。
2. **我最容易混淆的一点是什么？**
   容易把“前端校验（如 HTML 下拉框限制）”当成有效的安全防护，实际上它可以被抓包轻易绕过。
3. **我本次作业中哪一段最需要教师复核？**
   任务 4 中对于“输入校验”不能替代“参数化查询”的解释，希望老师看看表述是否专业。
4. **如果重做一次，我会优先改哪一部分？**
   我会在机制分析里结合代码层面的 AST（抽象语法树）变化，把“结构被污染”的底层原理画得更细致一些。

收口项 3：AI 输出审核与人工复核记录（必做）

1. **本作业哪些内容使用过 AI 辅助？**
   使用了 AI 辅助梳理“任务 4 防护清单”的表格排版，以及辅助生成了“关系图”的 Mermaid 代码。
2. **哪些判断是你自己复核后保留的？**
   AI 曾建议将 WAF 作为主要防护手段，但我复核后决定保留“参数化查询”作为唯一的【根因控制】手段，将 WAF 降级为【暴露面缩减】补充措施，这符合课程的防御层级逻辑。
3. **哪些内容你发现 AI 输出不准确并进行了修正？**
   AI 在分析统一错误处理时，曾错误地声称其能“切断盲注推断途径”。我人工将其修正为“仅能减少错误信息泄露，不能真正阻止布尔盲注或时间盲注”，确保了防御边界表述的绝对准确。
4. **你最终如何确认结论和证据是一一对应的？**

我严格对照了自查清单，确保每张 DVWA 截图下方，都附带了“这个差异支撑了什么判断”的因果说明，杜绝了有图无真相的现象。